Manufactured solutions for the monodomain equation

Construction of the exact fields, sources and reference functionals

Simão Castro

Table of contents

1 The problem **2**
1.1 Domain and governing equations . . . 2
1.2 Manufactured ionic closure . . . 3
1.3 Nomenclature . . . 3
1.4 Reading and regenerating . . . 5
1.5 Creating a new manufactured solution . . . 6

2 FDA cube **9**
2.1 Description and limitations . . . 9
2.2 Problem instance . . . 9
2.3 Construction . . . 9
2.4 Exact solution . . . 10
2.5 Self-checks . . . 10
2.6 Figures . . . 11
2.7 Regeneration . . . 12

3 Rotated anisotropic cube **12**
3.1 Description and limitations . . . 12
3.2 Problem instance . . . 12
3.3 Construction . . . 12
3.4 Exact solution . . . 13
3.5 Self-checks . . . 13
3.6 Figures . . . 13
3.7 Regeneration . . . 14

4 Sealed spherical shell **14**
4.1 Description and limitations . . . 14
4.2 Problem instance . . . 14
4.3 Construction . . . 14
4.4 Exact solution . . . 16
4.5 Self-checks . . . 16
4.6 Figures . . . 17
4.7 Regeneration . . . 18

5 Pseudo-ECG **18**
5.1 Description and limitations . . . 18

5.2 Problem instance . . . 18
5.3 Construction . . . 18
5.4 Exact solution . . . 19
5.5 Self-checks . . . 19
5.6 Figures . . . 20
5.7 Regeneration . . . 20

6 Purkinje graph **21**
6.1 Description and limitations . . . 21
6.2 Problem instance . . . 21
6.3 Construction . . . 21
6.4 Exact solution . . . 22
6.5 Self-checks . . . 22
6.6 Figures . . . 22
6.7 Regeneration . . . 23

7 Coupled 1D–3D junction **23**
7.1 Description and limitations . . . 23
7.2 Problem instance . . . 23
7.3 Construction . . . 23
7.4 Exact solution . . . 24
7.5 Self-checks . . . 24
7.6 Figures . . . 25
7.7 Regeneration . . . 25

References **25**

1 The problem

This document describes how each manufactured solution of the monodomain equation used by cardiacFoam is built. In the method of manufactured solutions (Roache 2002) an exact field is chosen first and the source that makes it satisfy the equation is derived from it. The document gives the exact fields, the sources that make them exact, the reference values of the derived functionals and the checks that each construction passes before anything is written. It states the problems only; it contains no solver output. Every displayed field, source and coefficient below is printed by the sympy generator that also writes the NumPy reference module and, for cardiacFoam, the C++ reference header, so the document, the references and the solver cannot disagree. A reader with this document and the generator folder can rebuild each problem in another code.

1.1 Domain and governing equations

In the myocardium $\Omega \subset \mathbb{R}^d$, $d \in \{1, 2, 3\}$, with sealed boundary $\partial\Omega$ and outward unit normal $\mathbf{n}$, the monodomain equation is

$$\begin{cases} \chi C_m \dfrac{\partial V_m}{\partial t} = \nabla \cdot (\boldsymbol{\sigma} \nabla V_m) - \chi C_m I_{\text{ion}} + I_{\text{app}} & \text{in } \Omega, \\ \mathbf{n} \cdot (\boldsymbol{\sigma} \nabla V_m) = 0 & \text{on } \partial\Omega, \end{cases}$$

with the ionic closure

$$I_{\text{ion}} = I(V_m, \mathbf{w}), \qquad \frac{\mathrm{d}\mathbf{w}}{\mathrm{d}t} = \mathbf{F}(V_m, \mathbf{w}),$$

and initial data $V_m(\mathbf{x}, 0)$, $\mathbf{w}(\mathbf{x}, 0)$. Every manufactured problem below prescribes the exact fields at $t = 0$ as initial data, and the zero conormal flux on every wall is satisfied by the exact field itself. The applied current I_{app} carries the manufactured source S where a case needs one.

The continuum equations use SI units: potentials in V, time in s, length in m, χ in m^{-1}, C_m in F m^{-2} and conductivity in S m^{-1}; I_{ion} is capacitance-normalised (V s^{-1}) and I_{app} is in A m^{-3}. The manufactured fields are written in case units on the normalised coordinates $x/(1\,\mathrm{m})$ and $t/(1\,\mathrm{s})$, so the arguments of the trigonometric, exponential and square-root functions are dimensionless.

1.2 Manufactured ionic closure

Every case uses the manufactured ionic model of Pathmanathan and Gray (2014), with state (V_m, u_1, u_2, u_3). The solver supplies V_m; the model advances the auxiliary states with V_m held fixed, at the rates

$$\dot{u}_1 = u_2^2\,(V_m - u_3)\left(-\frac{V_m}{2} + \frac{u_1}{2} + \frac{u_3}{2}\right) + u_2^2\,(-V_m + u_1 + u_3)^2$$

$$\dot{u}_2 = u_2^3\,(V_m - u_1 - u_3)$$

and $\dot{u}_3 = 0$. The model returns the current before capacitance normalisation,

$$J_{\text{ion}} = \frac{\beta\,(V_m - u_3)}{\chi} - \frac{C_m u_2^2\,(V_m - u_3)\,(-V_m + u_1 + u_3)}{2}$$

and the myocardium solver divides it by C_m,

$$I_{\text{ion}} = \frac{J_{\text{ion}}}{C_m}$$

because the equation carries $-\chi C_m I_{\text{ion}}$. The constants, with β_d the reaction coefficient of dimension d, are

$$C_m = 2, \quad \chi = 3, \quad \beta_1 = -1.1, \quad \beta_2 = -5.9, \quad \beta_3 = -8.6$$

Each case sets u_1 and u_2 from its own spatial factor so that the two rates hold exactly on the exact fields (Section 2, Section 4).

1.3 Nomenclature

The code name is the key of the cardiacFoam case dictionaries or the name used in the generators and the NumPy reference modules. “Case units” marks a quantity defined only through the manufactured construction, with no physical calibration.

Symbol	Meaning	Unit	Code name
$\mathbf{x} = (x, y, z)$	position, normalised by 1 m	m	`x`, `y`, `z`
d, Ω, $\mathbf{n}$	spatial dimension; the domain; outward unit normal of $\partial\Omega$	–; –; –	`dim`, `d`
t	time, normalised by 1 s	s	`t`
V_m	transmembrane potential	V	`Vm`
$\boldsymbol{\sigma}$	conductivity tensor of the monodomain equation	S m^{-1}	`conductivity`; `G` (shell generators), `K` (rotated cube), `sigma` (NumPy)
χ	membrane surface-to-volume ratio	m^{-1}	`chi`
C_m	membrane capacitance per unit area	F m^{-2}	`cm`
I_{ion}	capacitance-normalised ionic current, J_{ion}/C_m	V s^{-1}	(solver side)
J_{ion}	ionic current returned by the manufactured ionic model	A m^{-2} (case units)	`Iion`
I_{app}, S	applied volumetric current; the manufactured source	A m^{-3}	`source`
$\mathbf{w}$, $\mathbf{F}$	state vector of a general ionic model; its rate	case units	–
u_1, u_2, u_3	auxiliary states of the manufactured ionic model	case units	`u1`, `u2`, `u3`
β, β_d	reaction coefficient of the manufactured ionic model in dimension d	case units	`Beta`, `BETA`
F_d, G_d	spatial factors of V_m and of the auxiliary states	case units	`F`, `G` (FDA generator)
k_d	eigenvalue of $-\nabla\cdot(\boldsymbol{\sigma}\nabla\cdot)$ for F_d	s^{-1} (case units)	`eigenvalue(d)`
r_1, r_2	inner and outer radius of the shell	m	`r1`, `r2`
λ, ω	transmural coordinate; function vanishing on both spheres	–	`lam`, `om`
$\mathbf{f}, \mathbf{s}, \mathbf{n}_s$	fibre, sheet and sheet-normal directions	–	(shell generator)
$\mathbf{e}_a, \mathbf{e}_b$	fixed frame of the shell fibre rotation	–	(shell generator)
$\sigma_f, \sigma_s, \sigma_n$	conductivities along $\mathbf{f}$, $\mathbf{s}$, $\mathbf{n}_s$	S m^{-1}	`gil`, `git`, `gin`
θ_0, θ_1	fibre angle at the inner wall and its transmural increment	deg	`theta0`, `theta1`
γ_1, γ_2	rotation angles of the fixed frame	deg	`gamma1`, `gamma2`
$\hat{h}$, a_1, a_2, a_3, k	unsealed shell field and its coefficients	case units	`hhat`, `hA[0..2]`, `k`

Symbol	Meaning	Unit	Code name
m_j, m, d_j, $\mathbf{n}_j$	wall correction of sphere j, its blend, $\partial_n\omega$ on sphere j, unit normal of sphere j out of the heart	–	(shell generator)
g	positive spatial factor of the shell auxiliary states, in place of G_d (shell only)	case units	`g`
$\mathbf{r}_e$, Φ_e	electrode position; pseudo-ECG functional	m; case units	`electrodePositions`; `pseudoECG`
$\mathbf{r}$, φ_d	$\mathbf{x}-\mathbf{r}_e$; integrand of Φ_e in dimension d	m; case units	`integrand_d<d>`
c_{ECG}	normalisation of the pseudo-ECG functional, here 1	–	–
q; μ, ϑ	Gauss–Legendre points of the shell quadrature; $\cos\vartheta$; polar angle	–	–
q_{ref}	Gauss–Legendre points per axis of the reference quadrature	–	`referenceQuadratureOrder`
V_{1D}, g_e	graph transmembrane potential; graph edge conductivity (graph only)	V; S m^{-1}	`Vm1D`; `purkinjeConductivity`
R_{1D}, $I_{\mathrm{app},1D}$	residual of the graph equation; terminal term of the graph	case units	`residual_1D`; –
$\mathbf{x}_{\mathrm{root}}$, $\mathbf{x}_k$, L_e	graph root; terminal k; length of edge e	m	`ROOT`, `TERMINALS`
ρ, V_k	Purkinje fibre radius; volume of graph node k	m; m^3	`purkinjeFibreRadius`
R_{PVJ}, r_{PVJ}	junction resistance; kernel radius	case units; m	`rPvj`; `pvjRadius`
$I_{\mathrm{PVJ},k}$, S_{PVJ}	junction current of terminal k; junction source	case units	`continuum_current`; `continuum_junction_source`
w_k, B_k, $\bar{V}_k$	linear kernel of terminal k; ball of radius r_{PVJ} about $\mathbf{x}_k$; sampled tissue potential	–; –; V	`pvjKernel linear`; –; `continuum_sampling`

1.4 Reading and regenerating

The generators live in `tutorials/manufacturedSolutions/generators/` of the cardiacFoam repository. Each case has one sympy generator `<case>_mms.py` that writes

- `reference/<case>.py`, a NumPy-only module with the exact fields, sources and, where a functional is involved, its reference quadrature;
- `latex/<case>_*.tex`, the fragments printed in this document.

Set up the environment once and run a generator from that folder:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python fda_monodomain_mms.py --emit
.venv/bin/python fda_monodomain_mms.py --self-check
```

`--emit` runs the self-checks first and writes nothing if one fails; `--self-check` runs them alone. The generators that own a cardiacFoam header also take `--check PATH` and `--write PATH`; these compare or rewrite the C++ header and are needed only by cardiacFoam developers. The document itself is built with

```
quarto render docs/monodomain/monodomain_mms.qmd --to pdf
```

and its figures with `python docs/monodomain/figures.py`. Each case section ends with the commands of its generator.

1.5 Creating a new manufactured solution

A new manufactured solution is a new generator in `tutorials/manufacturedSolutions/generators/`. The steps below go from a copy of the template to a case that cardiacFoam runs and checks.

1.5.1 Copy the template

`template_mms.py` is the smallest complete generator: a passive monodomain field on the unit square with a diagonal conductivity and no ionic current,

$$V_m = \sqrt{t+1}\cos(\pi x)\cos(\pi y)$$

$$S = \left(\frac{\chi C_m}{2\sqrt{t+1}} + \pi^2(\sigma_x+\sigma_y)\sqrt{t+1}\right)\cos(\pi x)\cos(\pi y)$$

$$\chi = 3, \quad C_m = 2, \quad \sigma_x = 0.13, \quad \sigma_y = 0.0176$$

Copy it and its test, rename both, and edit every step marked `REPLACE`: `REFERENCE_NAME`, the coordinates, the constants, `exact_field`, `operator`, the self-checks and the emitted fragments.

```
cp template_mms.py mycase_mms.py
cp test_template_mms.py test_mycase_mms.py
```

1.5.2 Choose the exact field

Choose the field so that the boundary condition holds by construction; the solver's boundary treatment is then measured, not assumed. The generators of this folder use four patterns:

- a field whose gradient vanishes on every face, so every conormal flux is zero for any tensor (rotated cube, $\sin^2$ factors);
- a cosine eigenfunction of the operator with zero normal derivative on the faces of a box (FDA cube, the template);

- a sealing correction $F = \hat{h} + \omega m$ on a curved wall: ω vanishes on the walls and m cancels the conormal derivative of $\hat{h}$ there (the "Sealed spherical shell" section of this document);
- Dirichlet data plus pinned cells: the exact field is imposed on every face and on a layer of cells, so no flux condition is needed (Cartesian eikonal); this leaves the wall treatment untested.

Keep every factor smooth beyond the domain: polyhedral meshes put cell and face centres slightly outside a curved wall.

1.5.3 Derive the source

Write the operator the solver implements, with its signs and scalings, and take the source as its residual on the exact field, symbolically. For the monodomain equation that means the full tensor divergence $\nabla \cdot (\boldsymbol{\sigma} \nabla V_m)$, including the derivatives of a spatially varying tensor, the χC_m scaling of the time derivative, and the capacitance-normalised ionic current $I_{\text{ion}} = J_{\text{ion}}/C_m$ that enters as $-\chi C_m I_{\text{ion}}$. A source derived for a different convention converges to the wrong field.

1.5.4 Write the self-checks first

Each check fails before anything is written. The traps met in this folder, with the check that catches each:

trap	check
residual nonzero because a constant is rounded	keep constants as exact rationals; where a rounded constant is part of the problem, check the residual against a stated tolerance (FDA β_d)
conormal flux not zero on a curved wall	evaluate $\mathbf{n} \cdot \boldsymbol{\sigma} \nabla F$ on each wall at high precision (30 digits)
denominator that crosses zero	minimum of every denominator over the domain, extended past the walls, is positive
eikonal critical points	locate them; the Morse count (minima − saddles + maxima) on each sphere is 2 and every point is non-degenerate
electrode or quadrature node on a singular point	a coincident point contributes zero, with the solver's `VSMALL` guard reproduced in the generator
float sums that change with the Python version	`math.fsum` for every sum of floats
cancellation in a quadrature or finite-difference check	scale the tolerance by the sum of the absolute terms, not by the result
printed constant that hides digits	`printed_constant(value, digits)` asserts the printed decimal round-trips

Add a finite-difference evaluation of the operator on the exact field and compare it with the source; it catches an operator written for the wrong convention, which the symbolic residual cannot.

1.5.5 Emit and document

`python mycase_mms.py --emit` writes `reference/mycase.py` and `latex/mycase_*.tex`. Add a section to the document of the equation by copying an existing case section and keeping its order: Description and limitations, Problem instance, Construction, Exact solution, Self-checks, Figures, Regeneration. Print every formula by including its fragment from `../../latex/`, as the existing sections do; the release test fails if an emitted fragment is included nowhere. Add the figures as a function of `docs/<equation>/figures.py`, add the generator to `GENERATORS` of `test_release.py` and to `CLI_GENERATORS` of `test_mms_codegen.py`, and render:

```
python docs/monodomain/figures.py
quarto render docs/monodomain/monodomain_mms.qmd --to pdf
```

1.5.6 Use it in cardiacFoam

1. Write or extend a verifier in `src/verificationModels/<equation>Verification/` that evaluates the reference on the mesh and reports the error norms.
2. Generate its reference header with `--write PATH`: either a whole header, or the lines between `// BEGIN GENERATED <name>` and `// END GENERATED <name>` of a hand-written one. Map the header name to its region function in `HEADERS` of the generator (`HEADERS = {}` in the template).
3. Add the case to `parity/mmsParity.C` and `CASES` of `parity/test_parity.py`, and a `test_parity_<case>` to the generator test that compares the compiled header with `reference/mycase.py`.
4. Add a tutorial case in `tutorials/manufacturedSolutions/` with `Allrun`, a `regression/regressionTest.sh` on `tutorials/regression/lib.sh` and its reference, and mark it in the case table of `tutorials/README.md`.
5. Check the header, the parity and the regression:

   ```
   python mycase_mms.py --check PATH
   python -m pytest parity/test_parity.py test_mycase_mms.py
   ```

   then run `regression/regressionTest.sh` of the tutorial case.

1.5.7 Reproduce

A new generator is reproduced like the existing ones: set up the environment, run `--self-check` and `--emit`, the tests, the slow release test and the renders, as in the previous section. The release test then runs the new generator from a fresh environment and checks that its emitted files are unchanged.

2 FDA cube

2.1 Description and limitations

The monodomain problem of the FDA regulatory science tool for cardiac electrophysiology (Pathmanathan and Gray 2014; U.S. Food and Drug Administration, Center for Devices and Radiological Health 2023) on the unit cube $[0,1]^d$, $d = 1, 2, 3$, with a diagonal conductivity. It is the baseline manufactured solution of the verification literature; its role within the credibility framework for computational models is described by U.S. Food and Drug Administration (2023). The construction is summarised here and the reader is referred to Pathmanathan and Gray (2014) for its derivation.

The case exercises diffusion with a diagonal tensor, the ionic coupling and the time integration in one, two and three dimensions. The exact field is a product of cosines whose normal derivative vanishes on every face, and the tensor is aligned with the faces, so the zero-flux condition holds because of the field and the face normals are eigenvectors of $\boldsymbol{\sigma}$. The case therefore does not exercise a boundary flux reconstruction, a conormal flux that differs from the normal derivative, or off-diagonal conductivity.

The residual of the equation is not zero. The reaction coefficient β_d is given to two significant digits and the conductivity is tuned to nine digits so that the diffusion eigenvalue k_d equals $-\beta_d$ to that precision; the residual is exactly $(k_d + \beta_d)V_m$, and relative to $|\nabla \cdot (\boldsymbol{\sigma}\nabla V_m)| = k_d|V_m|$ it is

$$\frac{|k_1 + \beta_1|}{k_1} = 5.9 \times 10^{-11}, \quad \frac{|k_2 + \beta_2|}{k_2} = 2.5 \times 10^{-9}, \quad \frac{|k_3 + \beta_3|}{k_3} = 2.7 \times 10^{-9}$$

2.2 Problem instance

Unit cube $[0,1]^d$, all faces sealed, χ, C_m and β_d as in Section 1.2, and the diagonal conductivity

$$\boldsymbol{\sigma} = \begin{bmatrix} 0.111453302 & 0 & 0 \\ 0 & 0.12158542 & 0 \\ 0 & 0 & 0.030396355 \end{bmatrix}$$

In cardiacFoam:

- tutorial `monodomainPseudoECG`, options `-field fda` and `-dim 1|2|3`;
- ionic model `monodomainFDAManufactured`;
- verifier `manufacturedFDAMonodomainVerifier`.

2.3 Construction

The spatial factor F_d is a product of cosines, an eigenfunction of the diagonal operator: $\nabla\cdot(\boldsymbol{\sigma}\nabla F_d) = -k_d F_d$ with k_d the sum over the first d axes of $\sigma_{mm}(m\pi)^2$,

$$k_3 = \pi^2\sigma_{xx} + 4\pi^2\sigma_{yy} + 9\pi^2\sigma_{zz}$$

(k_1 and k_2 keep the first one and two terms). The time factor $\sqrt{1+t}$ and the auxiliary states are chosen so that $(u_1 - V_m)u_2^2 = 1/(1+t)$: the time derivative and the ionic current then collapse

into terms proportional to V_m, and the diffusion term into $-k_d V_m$, so that with $\beta_d = -k_d$ no applied current is needed. With β_d rounded to two digits the residual is

$$\chi C_m \frac{\partial V_m}{\partial t} - \nabla\cdot(\boldsymbol{\sigma}\nabla V_m) + \chi C_m I_{\text{ion}} = V_m\,(\beta_d + k_d)$$

which is the rounding-level term stated above; no source is applied ($I_{\text{app}} = 0$).

2.4 Exact solution

$$F_1 = \cos(\pi x)$$

$$F_2 = \cos(\pi x)\cos(2\pi y)$$

$$F_3 = \cos(\pi x)\cos(2\pi y)\cos(3\pi z)$$

$$G_1 = x + 1$$

$$G_2 = xy^2 + 1$$

$$G_3 = xy^2z^3 + 1$$

$$V_m = F_d\sqrt{t+1}$$

$$u_1 = G_d\,(t+1) + V_m$$

$$u_2 = \frac{1}{\sqrt{G_d\,(t+1)}}$$

$$u_3 = 0$$

The full expressions are `fields(d)` of `fda_monodomain_mms.py` and `Vm_d<d>`, `u1_d<d>`, `u2_d<d>`, `u3_d<d>`, `Iion_d<d>` (the current J_{ion}), `F_d<d>`, `G_d<d>` of `reference/fda_monodomain.py`.

2.5 Self-checks

`--self-check` asserts, for $d = 1, 2, 3$:

- $\partial_t u_1$, $\partial_t u_2$ and $\partial_t u_3$ equal the ionic rates on the exact fields (symbolic simplification to zero);
- the PDE residual equals $(k_d + \beta_d)V_m$ symbolically, and $|k_d + \beta_d|/k_d \le 10^{-8}$;
- $\mathbf{n}\cdot\nabla V_m$ vanishes on every face of the cube.

2.6 Figures

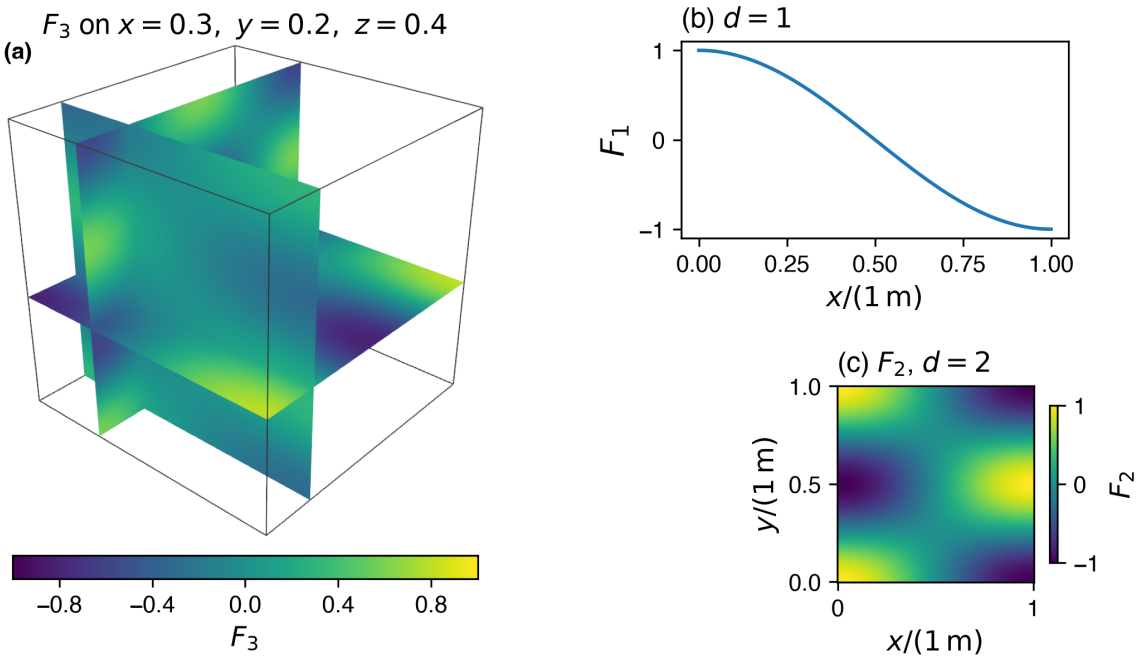
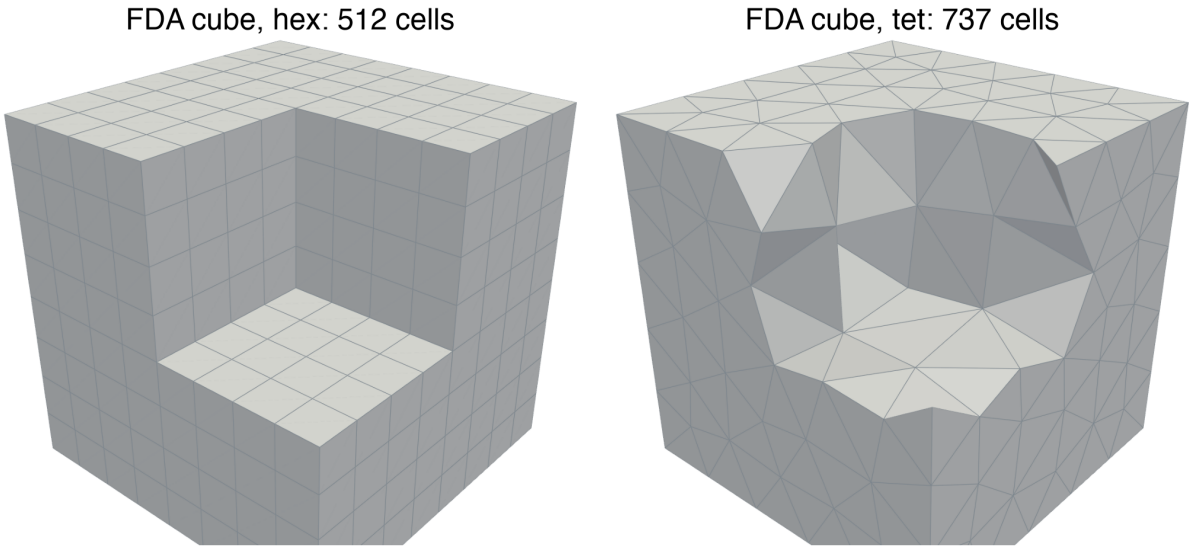


Figure 1: The FDA spatial factors: (a) F_3 on three orthogonal slices of the unit cube. (b) F_1 of $d = 1$. (c) F_2 of $d = 2$ on the unit square.



(a) hex: `-mesh hex -N 8`

(b) tet: `-mesh tet -N 5`

Figure 2: Meshes of the unit cube from the tutorial `monodomainPseudoECG` (`-dim 3`), made by the meshing steps of its `Allrun` at a coarse resolution chosen for legibility (the tutorial default is $N = 20$): (a) `blockMesh` with $N = 8$ cells per unit length; (b) `gmsh` tetrahedra of element size 1/5, seed 1, converted by `gmshToFoam`. The octant facing the viewer is removed cell by cell. The rotated cube of Section 3 uses the same meshes.

2.7 Regeneration

```
python fda_monodomain_mms.py --emit
python fda_monodomain_mms.py --self-check
H=../../../src/verificationModels/monodomainVerification
python fda_monodomain_mms.py --check $H/manufacturedFDAReference.H
```

`--emit` writes `reference/fda_monodomain.py` and `latex/fda_*.tex`. `--check` and `--write` act on the generated regions (`// BEGIN GENERATED <name>` to `// END GENERATED <name>`) of the cardiacFoam header `manufacturedFDAReference.H`: F_d, G_d, V_m, u_1, u_2, u_3 and J_{ion}; the rest of the header is hand-written.

3 Rotated anisotropic cube

3.1 Description and limitations

A three-dimensional field on the unit cube with a constant conductivity tensor whose off-diagonal entries are all non-zero, so the tensor axes are not aligned with the mesh. The case exercises the full anisotropic diffusion operator, including the mixed second derivatives, the cell-gradient reconstruction with a full tensor, the non-orthogonal correction, the ionic coupling and an applied manufactured source.

The gradient of the spatial factor vanishes on every face of the cube, so the conormal flux $\mathbf{n} \cdot \boldsymbol{\sigma} \nabla V_m$ is zero because of the field and not because of the operator. The boundary flux reconstruction is therefore not exercised: a wrong wall treatment that returns a zero face flux for a zero face gradient is not detected by this case.

3.2 Problem instance

Unit cube $[0, 1]^3$, all faces sealed, $\chi = 3$, $C_m = 2$, $\beta = \beta_3$, and the tensor

$$\boldsymbol{\sigma} = \begin{bmatrix} 0.106875 & -0.0084931 & -0.022561 \\ -0.0084931 & 0.116682 & -0.0130256 \\ -0.022561 & -0.0130256 & 0.0398782 \end{bmatrix}$$

In cardiacFoam this is `monodomainPseudoECG` with `-field anisotropic` (the default) and the verifier `manufacturedAnisotropicMonodomainVerifier`.

3.3 Construction

F is a product of $\sin^2$ factors, so F and ∇F vanish on every face. The auxiliary states take the form of the FDA problem with G_3, so the ionic rates hold exactly and the ionic model contributes $-C_m \partial_t V_m + \beta V_m / \chi$ as in Section 2. F is not an eigenfunction of the anisotropic operator, so the remainder is supplied as an applied current S, the residual of the equation with no source; it contains the six second-derivative terms of $\nabla \cdot (\boldsymbol{\sigma} \nabla F)$ for the constant tensor.

3.4 Exact solution

$$F = \sin^2(\pi x)\sin^2(\pi y)\sin^2(\pi z)$$

$$G_3 = xy^2z^3 + 1$$

$$V_m = F\sqrt{t+1}$$

$$u_1 = G_3(t+1) + V_m$$

$$u_2 = \frac{1}{\sqrt{G_3}(t+1)}$$

with $u_3 = 0$, and the source added to the right-hand side

$$S = \left(-\nabla\cdot(\boldsymbol{\sigma}\nabla F) + F\beta\right)\sqrt{t+1}$$

The expanded source is `fields(K)["S"]` and `cxx_source(K)` of `rotated_monodomain_mms.py` and `S` of `reference/rotated_monodomain.py`.

3.5 Self-checks

`--self-check` asserts that $\boldsymbol{\sigma}$ is symmetric with non-zero off-diagonal entries and positive definite (leading principal minors); that $\partial F/\partial x$, $\partial F/\partial y$ and $\partial F/\partial z$ vanish on the faces normal to each; that the source equals the six-term Hessian sum written in the form of the C++ reference; and that the ionic rates of u_1 and u_2 hold on the exact fields.

3.6 Figures

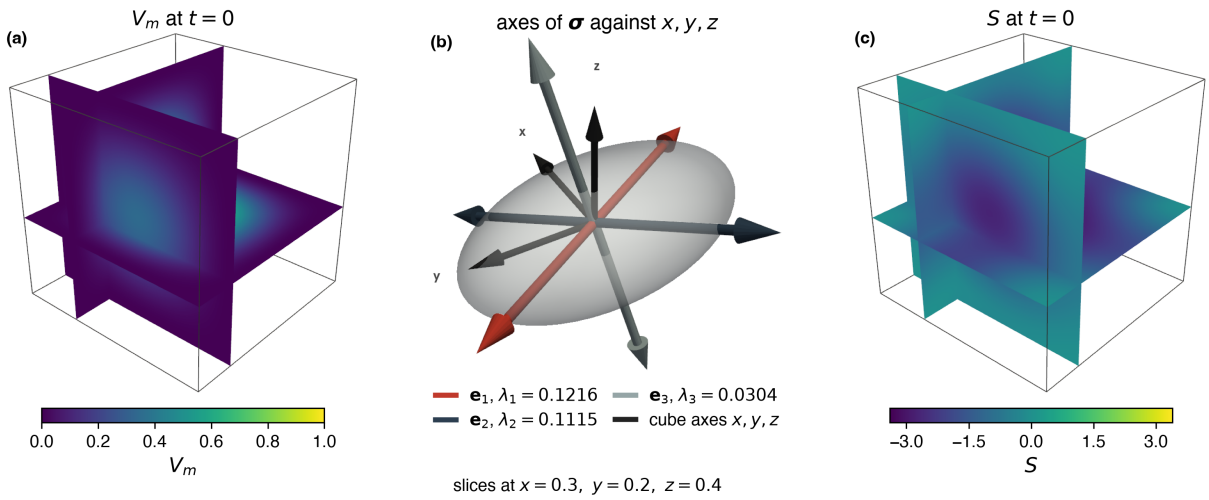


Figure 3: Rotated cube. (a) V_m at $t = 0$ on three orthogonal slices. (b) The ellipsoid of $\boldsymbol{\sigma}$ with its principal axes $\mathbf{e}_i$ (eigenvalues λ_i) against the cube axes. (c) The source S at $t = 0$ on the same slices.

3.7 Regeneration

```
python rotated_monodomain_mms.py --emit
python rotated_monodomain_mms.py --self-check
H=../../../src/verificationModels/monodomainVerification
python rotated_monodomain_mms.py --check \
    $H/manufacturedAnisotropicMonodomainReference.H
```

`--emit` writes `reference/rotated_monodomain.py` and `latex/rotated_*.tex`. The header regions are F, V_m, u_1, u_2, u_3 and S, with the tensor and β runtime arguments of the C++.

4 Sealed spherical shell

4.1 Description and limitations

A heart-like shell $r_1 < |\mathbf{x}| < r_2$ whose two spheres are sealed walls, with a full anisotropic conductivity whose fibre and sheet rotate through the wall. The tensor has three distinct eigenvalues and is oblique to both walls, and the exact field has non-zero normal and tangential gradients on both walls while its conormal flux vanishes there exactly. The case poses the full sealed-wall problem: curved walls, a rotating anisotropic tensor oblique to both walls, a zero conormal flux that is not a zero normal derivative, and a spatially varying tensor whose derivatives enter the source. Neither the construction nor its checks show a restriction beyond those of the stated problem.

4.2 Problem instance

The parameters are

$$\begin{gathered} r_1 = 0.5, \quad r_2 = 1, \quad \gamma_1 = 20^\circ, \quad \gamma_2 = 50^\circ, \quad k = \pi, \\ \sigma_f = 0.17, \quad \sigma_s = 0.019, \quad \sigma_n = 0.01, \quad \theta_0 = -20^\circ, \quad \theta_1 = 90^\circ, \\ a_1 = -0.1, \quad a_2 = 0.3, \quad a_3 = 0.25 \end{gathered}$$

with $\chi = 3$, $C_m = 2$ and $\beta = \beta_3$ (Section 1.2). The inner sphere is the patch `endocardium` and the outer sphere the patch `epicardium`. In cardiacFoam:

- tutorial `monodomainPseudoECG/sphericalShell`;
- verifier `manufacturedShellMonodomainVerifier`.

4.3 Construction

Transmural coordinate and conductivity. With λ running from 0 on the inner to 1 on the outer sphere, the fibre angle θ grows linearly through the wall. The fibre and sheet rotate by θ in the plane of a fixed orthonormal frame $(\mathbf{e}_a, \mathbf{e}_b, \mathbf{n}_s)$, the columns of $R_z(\gamma_1)R_x(\gamma_2)$. A fixed frame is used because a fibre field tangent to a sphere vanishes somewhere on it.

$$\lambda = \frac{-r_1^2 + |\mathbf{x}|^2}{-r_1^2 + r_2^2}$$

$$\theta = \lambda\theta_1 + \theta_0$$

$$[\mathbf{e}_a\ \mathbf{e}_b\ \mathbf{n}_s] = \begin{bmatrix} \cos(\gamma_1) & -\sin(\gamma_1) & 0 \\ \sin(\gamma_1) & \cos(\gamma_1) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\gamma_2) & -\sin(\gamma_2) \\ 0 & \sin(\gamma_2) & \cos(\gamma_2) \end{bmatrix}$$

$$\mathbf{f} = \mathbf{e}_a \cos(\theta) + \mathbf{e}_b \sin(\theta)$$

$$\mathbf{s} = -\mathbf{e}_a \sin(\theta) + \mathbf{e}_b \cos(\theta)$$

$$\boldsymbol{\sigma} = \mathbf{f} \otimes \mathbf{f}\sigma_f + \sigma_n \mathbf{n}_s \otimes \mathbf{n}_s + \sigma_s \mathbf{s} \otimes \mathbf{s}$$

Sealing. A smooth unsealed field $\hat{h}$ is corrected by a term that vanishes on both spheres. ω is zero on both spheres, with normal derivative d_j on sphere j; $\mathbf{n}_j$ is the unit normal pointing out of the heart.

$$\omega = \frac{\left(-r_1^2 + |\mathbf{x}|^2\right)\left(-r_2^2 + |\mathbf{x}|^2\right)}{(r_1 + r_2)\left(-r_1^2 + r_2^2\right)}$$

$$\mathbf{n}_1 = -\frac{\mathbf{x}}{r_1}$$

$$\mathbf{n}_2 = \frac{\mathbf{x}}{r_2}$$

$$d_1 = \frac{2r_1}{r_1 + r_2}$$

$$d_2 = \frac{2r_2}{r_1 + r_2}$$

On sphere j, $\nabla(\omega m) = m\, d_j \mathbf{n}_j$, so the correction m_j below cancels the conormal flux of $\hat{h}$ exactly. The blend m takes the value m_j on sphere j:

$$m_j = -\frac{\left(\mathbf{n}_j \cdot \boldsymbol{\sigma} \nabla \hat{h}\right)}{(\mathbf{n}_j \cdot \boldsymbol{\sigma}\, \mathbf{n}_j)\, d_j}$$

$$m = \lambda m_2 + m_1 (1 - \lambda)$$

$$F = \hat{h} + m\omega$$

so that $\mathbf{n}_j \cdot \boldsymbol{\sigma} \nabla F = 0$ on both spheres for every $\hat{h}$. The denominators $\mathbf{n}_j \cdot \boldsymbol{\sigma} \mathbf{n}_j$ are positive because $\boldsymbol{\sigma}$ is positive definite. Every factor is a polynomial, trigonometric function or ratio with a positive denominator, so the field extends smoothly beyond each sphere, where polyhedral mesh faces put cell and face centres.

Ionic closure and source. The auxiliary states use a positive function g in place of G_d, so the ionic rates hold exactly; the ionic model then contributes $-C_m\partial_t V_m + \beta V_m/\chi$ and the applied current is the remainder S, in which $\nabla\cdot(\boldsymbol{\sigma}\nabla F)$ includes the derivatives of the spatially varying tensor.

4.4 Exact solution

$$\hat{h} = a_1 z + a_2\left(1 - \frac{3x^2}{10}\right)\cos(ky) + a_3 x\sin(kz)$$

$$V_m = F\sqrt{t+1}$$

$$g = \frac{\sin\left(kx + \frac{1}{5}\right)\cos(kz)\cos\left(ky - \frac{1}{3}\right)}{4} + 1$$

$$u_1 = V_m + g\,(t+1)$$

$$u_2 = \frac{1}{\sqrt{g\,(t+1)}}$$

with $u_3 = 0$ and the source

$$S = \sqrt{t+1}\,(F\beta - \nabla\cdot(\boldsymbol{\sigma}\nabla F))$$

$\nabla\cdot(\boldsymbol{\sigma}\nabla F)$ is not printed: it is `expressions()["divGGradF"]` of `shell_monodomain_mms.py` and `divSigmaGradF` of `reference/shell_monodomain.py`, which also holds `F`, `gradF`, `sigma` and `g` as functions of (x, y, z) and the parameters, and `field(name, x, y, z, **overrides)` to evaluate them with the default parameters.

4.5 Self-checks

`--self-check` asserts:

- sealing: $\max|\mathbf{n}\cdot\boldsymbol{\sigma}\nabla F| < 10^{-25}$ at 200 points of each sphere, evaluated in 30-digit arithmetic on the exact sphere;
- the source: $\nabla\cdot(\boldsymbol{\sigma}\nabla F)$ agrees with central differences of the flux $\boldsymbol{\sigma}\nabla F$ to 10^{-6} at 40 points in the shell;
- positivity: $\min\mathbf{n}_j\cdot\boldsymbol{\sigma}\mathbf{n}_j > 0$ and $\min g > 0.5$ over 4000 points of the shell widened by 0.05 on each side;
- richness of the wall flux: over each wall, the mean of the tangential part of $\boldsymbol{\sigma}\mathbf{n}$ (divided by σ_f) and the mean of the tangential part of ∇F both exceed 10^{-2}. The tangential part of $\boldsymbol{\sigma}\mathbf{n}$ is a tangent field on a sphere and vanishes at isolated points, so its wall mean, not its minimum, measures how oblique the tensor is.

4.6 Figures

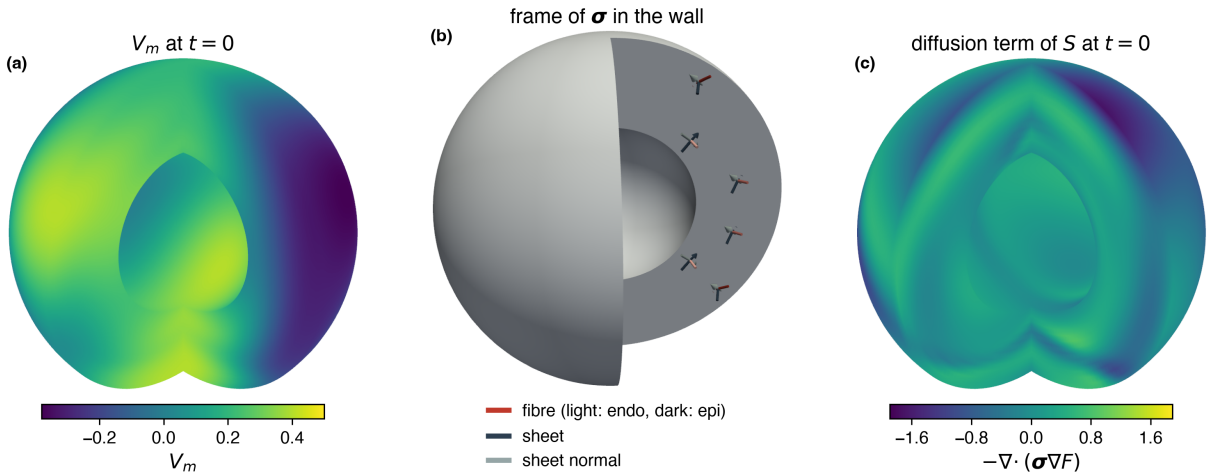
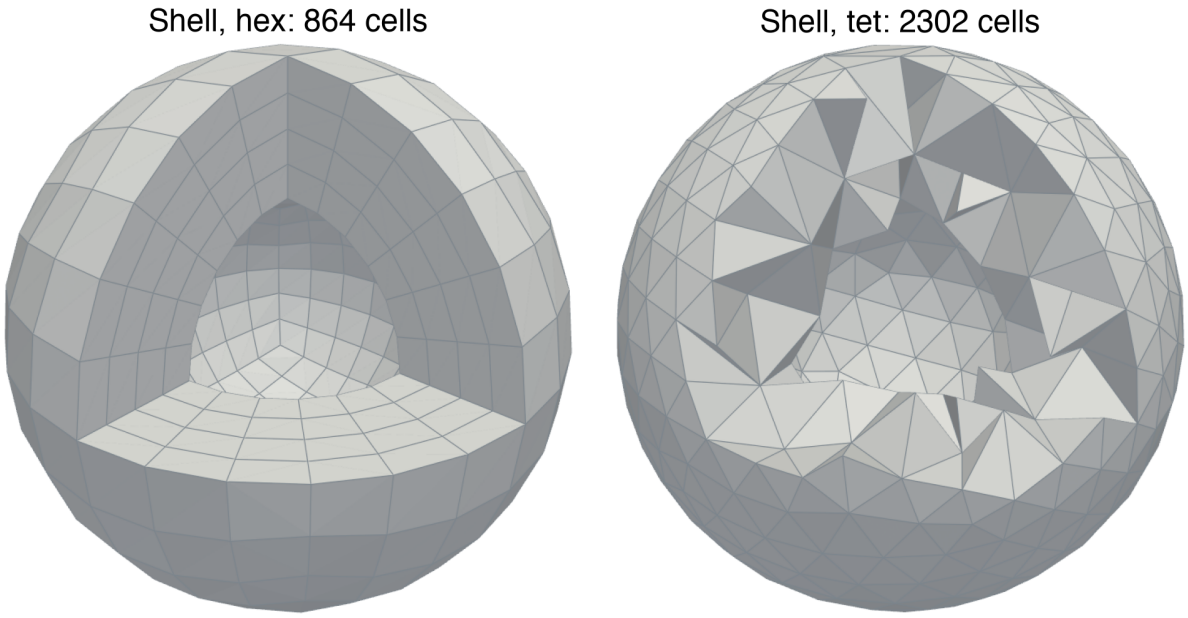


Figure 4: Sealed shell. (a) V_m at $t = 0$ on a cut-away of the shell. (b) The fibre, sheet and sheet-normal directions at six sites through the wall on the cut face, the fibre shaded from the inner (light) to the outer (dark) wall. (c) The diffusion term of the source as it enters S, $-\nabla \cdot (\boldsymbol{\sigma}\nabla F)$.



(a) hex: `-mesh hex -N 8`

(b) tet: `-mesh tet -N 5`

Figure 5: Meshes of the shell from the tutorial `monodomainPseudoECG/sphericalShell`, made by the meshing steps of its `Allrun` at a coarse resolution chosen for legibility (the tutorial default is $N = 12$): (a) the `blockMesh` cubed sphere with $N = 8$; (b) `gmsh` tetrahedra of element size 1/5, seed 1, converted by `gmshToFoam`. The octant facing the viewer is removed cell by cell, exposing the cavity.

4.7 Regeneration

```
python shell_monodomain_mms.py --emit
python shell_monodomain_mms.py --self-check
H=../../../src/verificationModels/monodomainVerification
python shell_monodomain_mms.py --check $H/manufacturedShellMonodomainReference.H
```

`--emit` writes `reference/shell_monodomain.py`, `latex/shell_*.tex` and `latex/shell_monodomain_*.tex`. `--write` writes the whole header, including a table of self-test samples that the cardiacFoam verifier compares with its compiled functions at start-up.

5 Pseudo-ECG

5.1 Description and limitations

The pseudo-ECG functional (Gima and Rudy 2002) of the cube fields, evaluated at electrodes outside the domain, with the normalisation $c_{\mathrm{ECG}} = 1$. The case exercises the reconstructed gradient of the computed potential and its volume quadrature against the functional of the exact field. The reference is a quadrature of the exact integrand, not a closed form: its accuracy is that of the quadrature at the electrodes used, and it inherits the limitations of the field it observes (Section 2, Section 3). The generator covers the cube fields. The shell pseudo-ECG reference of cardiacFoam (`domain shell`) is the same integrand over the shell; it is computed by the verifier from the shell header and is not part of the emitted modules.

5.2 Problem instance

The fields $V_m = \sqrt{1+t}\,F$ of Section 2 ($F = F_d$, diagonal tensor) and of Section 3 (F the $\sin^2$ factor, full tensor). The cardiacFoam tutorial `monodomainPseudoECG` uses 161 electrodes outside the unit cube and evaluates the pseudo-ECG in three dimensions.

5.3 Construction

For $V_m = \sqrt{1+t}\,F$ the pseudo-ECG separates into the time factor and a spatial integral. With $\mathbf{r} = \mathbf{x} - \mathbf{r}_e$ and, in $d < 3$, only the first d coordinates of $\mathbf{r}$ and ∇F,

$$\varphi_d = \frac{\mathbf{r} \cdot \boldsymbol{\sigma} \nabla F_d}{\|\mathbf{r}\|^3}$$

$$\Phi_e(\mathbf{r}_e, t) = \Phi_e(\mathbf{r}_e, 0)\sqrt{t+1}$$

$$\Phi_e(\mathbf{r}_e, 0) = \int_{[0,1]^d} \varphi_d \, \mathrm{d}\mathbf{x}$$

The gradients are

$$\nabla F_1 = \Big[-\pi \sin(\pi x)\Big]$$

$$\nabla F_2 = \begin{bmatrix} -\pi \sin(\pi x) \cos(2\pi y) \\ -2\pi \sin(2\pi y) \cos(\pi x) \end{bmatrix}$$

$$\nabla F_3 = \begin{bmatrix} -\pi \sin(\pi x) \cos(2\pi y) \cos(3\pi z) \\ -2\pi \sin(2\pi y) \cos(\pi x) \cos(3\pi z) \\ -3\pi \sin(3\pi z) \cos(\pi x) \cos(2\pi y) \end{bmatrix}$$

$$\nabla F = \begin{bmatrix} 2\pi \sin(\pi x) \sin^2(\pi y) \sin^2(\pi z) \cos(\pi x) \\ 2\pi \sin^2(\pi x) \sin(\pi y) \sin^2(\pi z) \cos(\pi y) \\ 2\pi \sin^2(\pi x) \sin^2(\pi y) \sin(\pi z) \cos(\pi z) \end{bmatrix}$$

5.4 Exact solution

The reference is a tensor-product Gauss–Legendre quadrature with q_{ref} points per axis on $[0,1]^d$; at a quadrature point closer to the electrode than the guard the integrand is set to zero:

$$q_{\mathrm{ref}} = 96, \quad \varphi_d\big|_{\|\mathbf{r}\|^2 \le 10^{-300}} = 0, \quad \varphi_1\big|_{|x - r_{e,x}| \le 10^{-300}} = 0$$

Note for implementers. The tutorials use $q_{\mathrm{ref}} = 96$ for the pseudo-ECG. The guard makes an electrode that coincides with a quadrature point contribute 0, not a non-finite value; a reference built in another code must use the same convention to agree at such an electrode. In one dimension the guard is $|x - r_{e,x}| \le 10^{-300}$, as printed above. The NumPy module holds `integrand_d1`, `integrand_d2`, `integrand_d3`, `integrand_rotated` (arguments x, y, z, the electrode and the six distinct entries of $\boldsymbol{\sigma}$), `spatial_reference(case, sigma, electrode, order)`, `reference(case, sigma, electrode, t, order)` and `time_factor(t)`, with `case` one of `"d1"`, `"d2"`, `"d3"`, `"rotated"`. For the shell, cardiacFoam integrates the same kernel with q Gauss–Legendre nodes in the radius $|\mathbf{x}|$ (Jacobian $|\mathbf{x}|^2$) and in $\mu = \cos\vartheta$, with ϑ the polar angle, and $2q$ trapezoid nodes in the azimuth, and requires every electrode to lie outside r_2.

5.5 Self-checks

`--self-check` asserts that the integrand equals the Cartesian component sums of the C++ reference for $d = 1, 2, 3$, for the diagonal and the rotated tensor, at two points and electrodes, to 10^{-12}; and that the integrand vanishes at an electrode placed on the evaluation point.

5.6 Figures

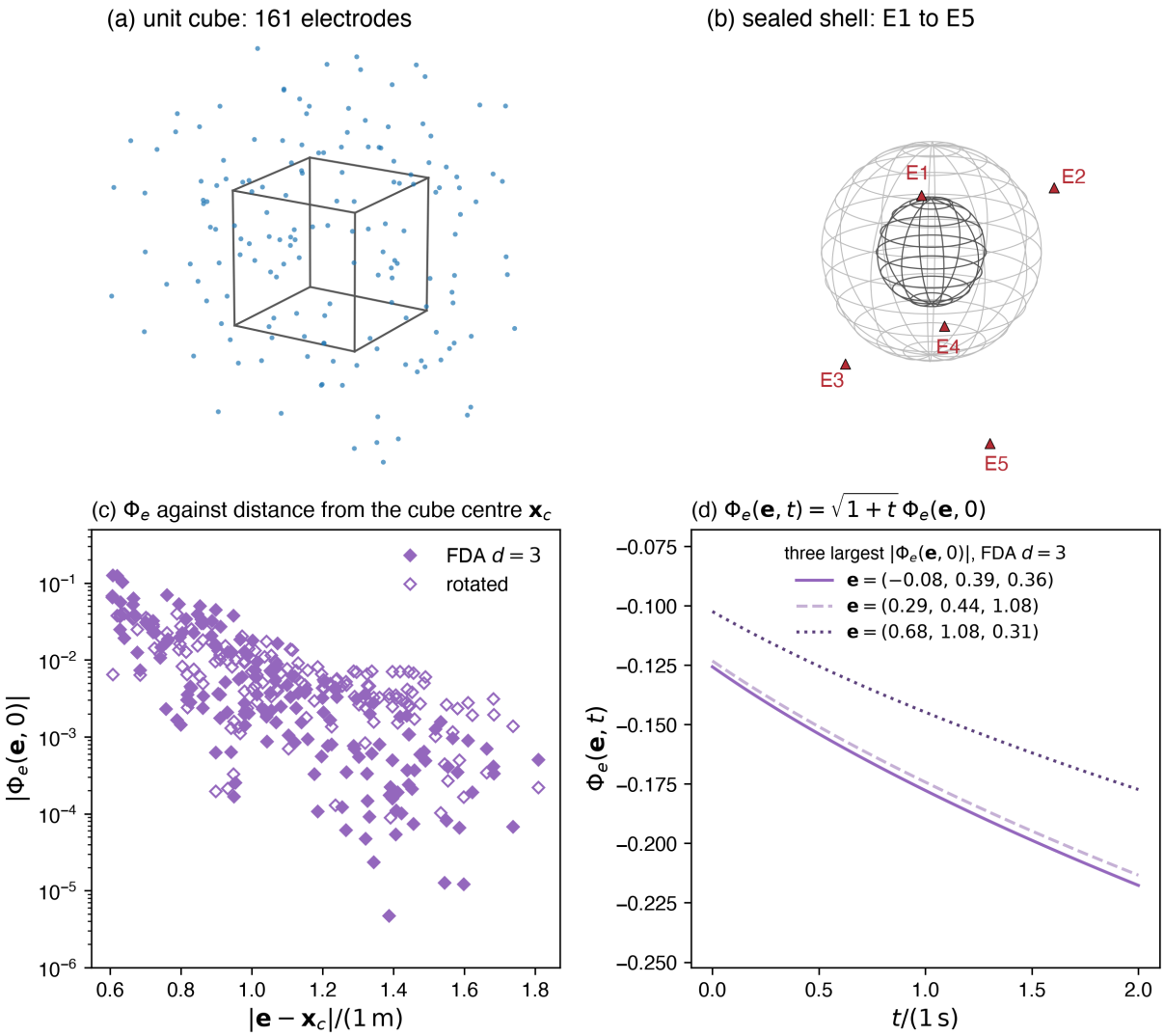


Figure 6: Pseudo-ECG. (a) The 161 electrodes of the cube tutorial around the unit cube. (b) The five named electrodes of the shell tutorial. (c) The spatial reference $\Phi_e(\mathbf{r}_e, 0)$ against the distance of the electrode from the cube centre. (d) $\Phi_e(\mathbf{r}_e, t) = \sqrt{1+t}\,\Phi_e(\mathbf{r}_e, 0)$ at the three electrodes of largest $|\Phi_e(\mathbf{r}_e, 0)|$, for the FDA field with $d = 3$. The figure writes $\mathbf{e}$ for the electrode position $\mathbf{r}_e$.

5.7 Regeneration

```
python pseudo_ecg_mms.py --emit
python pseudo_ecg_mms.py --self-check
H=../../../src/verificationModels/monodomainVerification
python pseudo_ecg_mms.py --check $H/manufacturedFDAReference.H
python pseudo_ecg_mms.py --check $H/manufacturedAnisotropicMonodomainReference.H
```

`--emit` writes `reference/pseudo_ecg.py` and `latex/pseudo_ecg_*.tex`. The header regions are the time factor, ∇F and the kernel; the guards, the product $\boldsymbol{\sigma}\nabla F$ and the quadrature rule are hand-written.

6 Purkinje graph

6.1 Description and limitations

A reaction–diffusion Purkinje graph: two straight branches from a root to two terminals, carrying the one-dimensional FDA field along the arc coordinate. The case exercises the node–edge graph diffusion operator, the graph ionic coupling and, in the coupled problem (Section 7), the terminal term of bidirectional transfer. It has no branch point with more than two edges, so the flux balance at a junction of three or more edges is not exercised.

The graph solver applies no bulk manufactured source. The FDA closure makes the residual of the graph equation vanish up to the rounding-level term $(g_e\pi^2 + \beta_1)V_{1D}$, as in Section 2: the graph conductivity is the first diagonal entry of the FDA tensor, tuned to the eigenvalue of $\cos(\pi x)$.

6.2 Problem instance

$$g_e = 0.111453302, \quad \chi = 3, \quad C_m = 2, \quad \beta_1 = -1.1$$

$$\mathbf{x}_{\text{root}} = \left(\frac{1}{2}, \frac{1}{6}, \frac{1}{3}\right), \quad \mathbf{x}_1 = \left(0, \frac{1}{6}, \frac{1}{3}\right), \quad \mathbf{x}_2 = \left(1, \frac{1}{6}, \frac{1}{3}\right)$$

Root $\mathbf{x}_{\text{root}}$, terminals $\mathbf{x}_1$, $\mathbf{x}_2$ on the faces $x = 0$ and $x = 1$ of the tissue cube, each branch along x with arc coordinate equal to x. In cardiacFoam this is the graph of the tutorial `monodomain1D3D` (graph files `purkinjeGraph.nodes<k>`, edge length $1/(k-1)$), with the verifier `manufacturedGraphVerifier`; the tutorial sets the Purkinje fibre radius $\rho = 1/\sqrt{\pi}$ m, so that $\pi\rho^2 = 1$.

6.3 Construction

On each edge the graph equation is $\chi C_m \partial_t V_{1D} = \partial_x(g_e\, \partial_x V_{1D}) - \chi C_m I_{\text{ion},1D}$. The field is $V_{1D} = \sqrt{1+t}\, F_1$ with the FDA auxiliary states of $d = 1$ (G_1), so the closure of Section 2 applies unchanged and its residual is

$$R_{1D} = V_{1D}\left(\beta_1 + \pi^2 g_e\right)$$

which is kept as a check and is not applied. The terminals are placed where the tissue field of Section 7 takes half the graph value with the opposite sign and has a zero x-derivative:

$$V_m(\mathbf{x}_k) = -\frac{V_{1D}}{2}$$

Bidirectional terminal term. In bidirectional transfer the terminal node k loses the junction current of Section 7 through the volume V_k of the node. Each edge gives half of $\pi\rho^2 L_e$ to each of its end nodes, so

$$V_k = \frac{\pi\rho^2 \sum_{e \ni k} L_e}{2}$$

and a terminal, a leaf, has $V_k = \pi\rho^2 L_e/2$. The graph receives

$$I_{\mathrm{app},1D}(\mathbf{x}_k) = -\frac{I_{\mathrm{PVJ},k}}{V_k}$$

at node k; in unidirectional transfer the term is zero.

6.4 Exact solution

$$V_{1D} = \sqrt{t+1}\cos(\pi x)$$

with u_1, u_2, u_3 of Section 2 for $d = 1$. The full expressions are `fields()` of `graph_monodomain_mms.py` and `V1D`, `u1`, `u2`, `u3`, `Iion`, `residual_1D` of `reference/graph_monodomain.py`.

6.5 Self-checks

`--self-check` asserts that the graph residual equals $(g_e\pi^2 + \beta_1)V_{1D}$ symbolically and that $|g_e\pi^2 + \beta_1| \le 10^{-8}|\beta_1|$; that g_e equals σ_{xx} of the FDA tensor; that the ionic rates of u_1 and u_2 hold; and that at both terminals $V_m = -V_{1D}/2$ and $\partial V_m/\partial x = 0$.

6.6 Figures

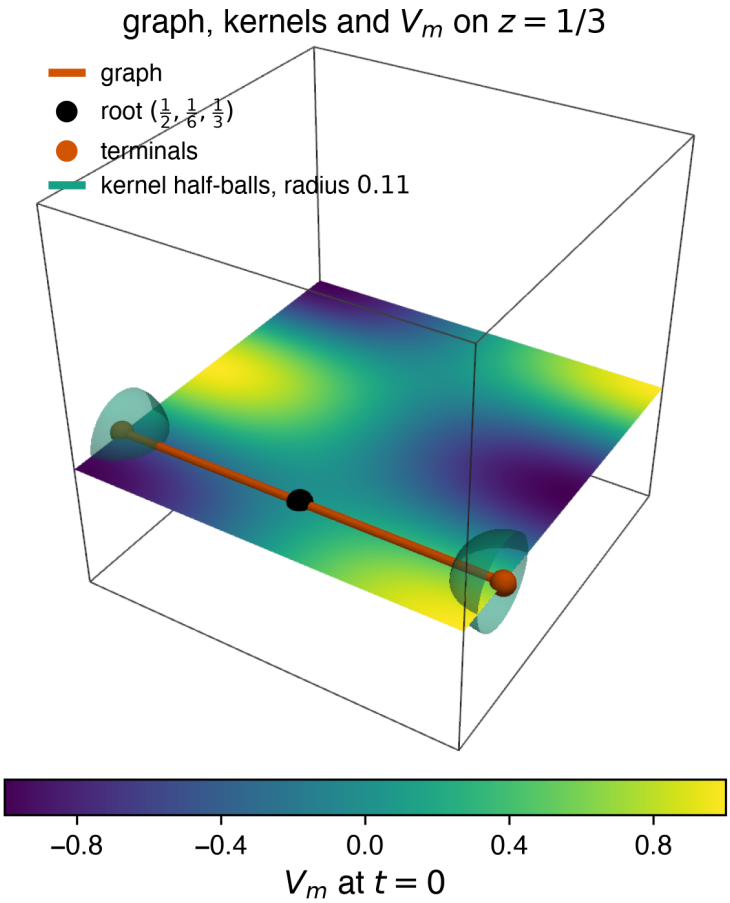


Figure 7: Graph and tissue geometry: the unit cube with the graph (root, two branches, two terminals on the faces $x = 0$ and $x = 1$), the half-ball kernels of radius r_{PVJ} at the terminals, and V_m of the tissue on the plane $z = 1/3$ at $t = 0$.

6.7 Regeneration

```
python graph_monodomain_mms.py --emit
python graph_monodomain_mms.py --self-check
```

`--emit` writes `reference/graph_monodomain.py` and `latex/graph_*.tex`. The generator owns no cardiacFoam header.

7 Coupled 1D–3D junction

7.1 Description and limitations

The Purkinje graph of Section 6 joined to the three-dimensional tissue of Section 2 ($d = 3$) at the two terminals by a resistive Purkinje–myocardial junction (PVJ) (Vergara et al. 2016). The tissue and the graph fields are chosen to differ at the terminals, so the junction currents are non-zero and known. The junction is defined here in continuum form: sampling of the tissue potential and deposition of the junction current with a linear kernel of finite radius about each terminal.

Limitation. Any implementation samples and deposits with its own discrete map of the kernel. An exact junction source built with the solver's own discrete map (as cardiacFoam does) checks the coupled workflow against that map, not the map against the continuum coupling defined below; the difference between the discrete map and the continuum integrals is a mesh-dependent consistency error. The NumPy module holds the continuum junction only and does not port a discrete map. The case has no calibrated junction resistance and no explicit junction delay.

7.2 Problem instance

Tissue: the FDA cube of Section 2 with $d = 3$. Graph: Section 6. Junction:

$$r_{\mathrm{PVJ}} = 0.11, \quad R_{\mathrm{PVJ}} = 1$$

In cardiacFoam this is the tutorial `monodomain1D3D`. Its coupler is `reactionDiffusionPvjCoupler` (`pvjKernel linear`, `pvjRadius`, `rPvj`) and its verifier `coupled1D3DMonodomainVerifier`.

7.3 Construction

The kernel of terminal k is linear in the distance and is restricted to the domain. Both terminals lie on a face of the cube, so its support $B_k \cap \Omega$ is a half ball:

$$w_k(\mathbf{x}) = \max\left(0, 1 - |\mathbf{x} - \mathbf{x}_k| / r_{\mathrm{PVJ}}\right)$$

$$\int_{B_k \cap \Omega} w_k \, \mathrm{d}\mathbf{x} = \frac{\pi r_{\mathrm{PVJ}}^3}{6}$$

Sampling. The tissue potential seen by terminal k is the kernel-weighted mean

$$\bar{V}_k = \frac{\int_{B_k \cap \Omega} w_k V_m \, \mathrm{d}\mathbf{x}}{\int_{B_k \cap \Omega} w_k \, \mathrm{d}\mathbf{x}}$$

Current.

$$I_{\mathrm{PVJ},k} = \frac{V_{1D,k} - \bar{V}_k}{R_{\mathrm{PVJ}}}$$

Deposition. Each terminal deposits its own current with its kernel normalised to unit integral, so the integral of the deposition of terminal k over the cube is $I_{\mathrm{PVJ},k}$:

$$S_{\mathrm{PVJ}}(\mathbf{x}) = \sum_{k=1}^{2} \frac{I_{\mathrm{PVJ},k} w_k(\mathbf{x})}{\int_{B_k \cap \Omega} w_k \, \mathrm{d}\mathbf{x}}$$

In bidirectional transfer the graph loses the same current at the terminal (Section 6).

7.4 Exact solution

At a terminal $V_m = -V_{1D}/2$ and $\partial_x V_m = 0$, so the point-value current is

$$I^{\mathrm{point}}_{\mathrm{PVJ},k} = \pm \frac{3\sqrt{t+1}}{2R_{\mathrm{PVJ}}}$$

with $+$ at $\mathbf{x}_1$ and $-$ at $\mathbf{x}_2$. The sampled value differs from the point value at second order in the radius: by the symmetry of the half ball in y and z and $\partial_x V_m = 0$ the first-order Taylor term vanishes, and a second-order remainder bound gives

$$\left| \bar{V}_k - V_m(\mathbf{x}_k) \right| \le 7\pi^2 r^2_{\mathrm{PVJ}} \sqrt{t+1}$$

The exact junction source on any mesh follows from `continuum_sampling`, `continuum_current`, `terminal_deposition` and `continuum_junction_source` of `reference/coupled_1d3d.py`. The half-ball integrals use Gauss–Legendre quadrature in spherical coordinates about the terminal, with the polar axis normal to the cube face and the polar cosine split where the face cuts the ball, so every piece has a smooth integrand (`half_ball_kernel_integral`, n points per direction and piece, $n = 32$ by default).

7.5 Self-checks

`--self-check` asserts, at $t = 0.07$, that the point-value currents are $\pm\frac{3}{2}\sqrt{1+t}$ and equal $V_{1D} - V_m$ at each terminal; that the sampled value $\bar{V}_k$ lies within the second-order bound above; and that the quadrature of the kernel mass equals $\pi r^3_{\mathrm{PVJ}}/6$ to 10^{-12} relative. The test file `test_coupled_1d3d_mms.py` also integrates each terminal's deposition against its current and measures the second-order approach of $\bar{V}_k$ to the point value.

7.6 Figures

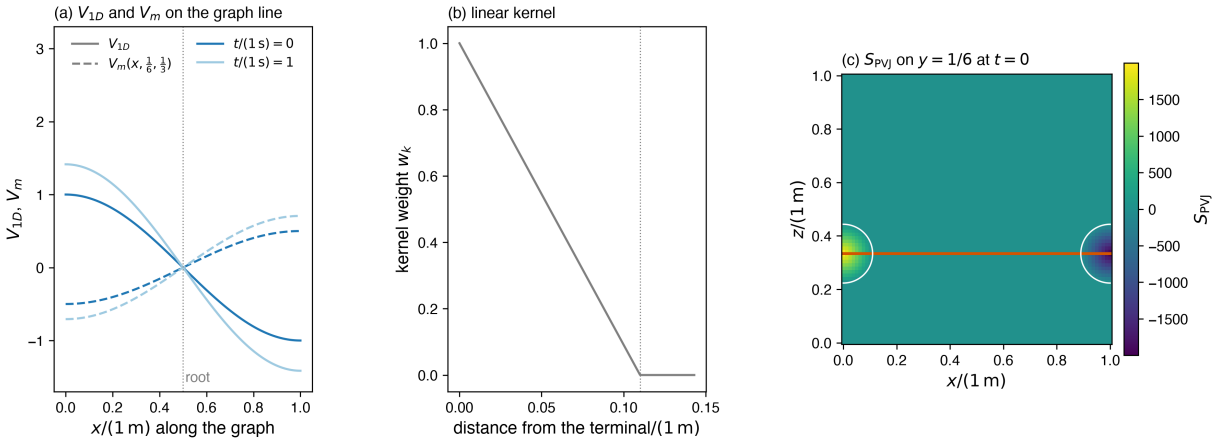


Figure 8: Graph and junction fields. (a) V_{1D} (solid) and the tissue V_m along the graph line (dashed) at two times. (b) The linear kernel profile w_k against the distance from the terminal. (c) The junction source S_{PVJ} on the plane $y = 1/6$ at $t = 0$, with $R_{\mathrm{PVJ}} = 1$.

7.7 Regeneration

```
python coupled_1d3d_mms.py --emit
python coupled_1d3d_mms.py --self-check
```

`--emit` writes `reference/coupled_1d3d.py` (with the half-ball quadrature, NumPy only) and `latex/coupled_*.tex`. The generator owns no cardiacFoam header; the cardiacFoam verifier builds its exact source with the solver's discrete map and is not generated.

References

Gima, Kazutaka, and Yoram Rudy. 2002. "Ionic Current Basis of Electrocardiographic Waveforms: A Model Study." *Circulation Research* 90 (8): 889–96. https://doi.org/10.1161/01.RES.0000016960.61087.86.

Pathmanathan, Pras, and Richard A. Gray. 2014. "Verification of Computational Models of Cardiac Electro-Physiology." *International Journal for Numerical Methods in Biomedical Engineering* 30 (5): 525–44. https://doi.org/10.1002/cnm.2615.

Roache, Patrick J. 2002. "Code Verification by the Method of Manufactured Solutions." *Journal of Fluids Engineering* 124 (1): 4–10. https://doi.org/10.1115/1.1436090.

U.S. Food and Drug Administration. 2023. "Assessing Credibility of Computational Modeling and Simulation in Medical Device Submissions: Guidance for Industry and Food and Drug Administration Staff." Guidance Document. Center for Devices and Radiological Health (CDRH). https://www.fda.gov/media/154985/download.

U.S. Food and Drug Administration, Center for Devices and Radiological Health. 2023. "Verification Test Problems for Cardiac Electrophysiology Modeling Software." Catalog of Regulatory Science Tools. https://cdrh-rst.fda.gov/verification-test-problems-cardiac-electrophysiology-modeling-software.

Vergara, Christian, Matthias Lange, Simone Palamara, Toni Lassila, Alejandro F. Frangi, and Alfio Quarteroni. 2016. "A Coupled 3D–1D Numerical Monodomain Solver for Cardiac Electrical Activation in the Myocardium with Detailed Purkinje Network." *Journal of Computational Physics* 308: 218–38. https://doi.org/10.1016/j.jcp.2015.12.016.